

Smart Assistive System – Deliverable 4 (Sprint 4 Report)

1. Document Title

Smart Assistive System – Deliverable 4 (Sprint 4 Report)

2. Team Name

Vision Assist Innovators

3. Project Name

Smart Assistive System – Vision Assistance & Environment Awareness

4. Students' Names and IDs

- Sarang Prajapati – N01662036
 - Krish Patel – N01666556
 - Kush Patel – N01657387
 - Daksh Rana – N01664095
-

5. Table of Contents

1. Document Title	
2. Team Name	
3. Project Name	
4. Students' Names and IDs	
5. Table of Contents	
6. Requirement + Comment Format	
7. Signatures Table	
8. Members Info and Participation	
9. Brief Description of the Project	
10. Issue the Product Solves	
11. Comparison with Two Existing Apps	
12. Differences Between Our App and the Two Apps	
13. Why Our App Is Better (for Our Scenario)	
14. Pros and Cons Table	
15. GitHub Repo Link	
16. Verify Link Is Working	
17. Login Credentials for Testing	
18. Other Login Credentials (Admin vs Regular)	
19. Sprint 4	
20. Work Done by Each Team Member (Sprint 4 Only)	
21. Minimum 10 Commits per Member (from Nov 3)	
22. Marks Deducted if Less than 10 Commits	
23. Commit Table for Sprint 4	
24. Sprint Goals	
25. Updated Sprint Dashboard (Sprint 4) Screenshot	

26. Dashboard Fields	
27. Tool for Board (Monday.com)	_
28. Use Scrum, Not Kanban	___
29. Screenshot Showing Stories + Tasks (Sprint 4 Only)	
30. Screenshots of Team Discussion (3 Different Dates)	
31. Sprint 4 Retrospective	
32. C4 System Context Diagram	
33. C4 Container Diagram	
34. Document Two Design Patterns with Code	
35. Design Principles and Patterns Considered	
36. Coding Progress Since Deliverable 3	
37. Unit Test Cases (≥10)	
38. Use of assertEquals, assertTrue, assertFalse, assertNotNull ...	
39. All Test Cases Pass	
40. Screenshot of Passing Tests	
41. Functionality on Customer Feedback Screen	
42. Invalid Input Handling	
43. Progress Bar & 5-Second Delay	
44. AlertDialog After DB Confirmation	
45. Device Model Stored in DB	
46. Screenshot – Progress Bar	
47. Screenshot – AlertDialog	
48. Screenshot – Data Stored in DB (≥3 Entries)	
49. Clear Input & Restrict to Once per 24 Hours	
50. Gray Out Submit Button & Show Timer	
51. Description of How Requirement 50–51 Is Met	

52. Screenshot Showing All Sensor Data Fetched & Updated

53. At Least Two Core Features Implemented

6. Requirement + Comment Format

From Item 9 onward, each section of this document follows the format:

- **Requirement:** (short version of professor text)
- **Answer:** Our explanation of how the app satisfies it.

7. Signatures Table

Name	Student ID	GitHub ID	Signature (on printed copy)	Effort (%)
Sarang Prajapati	N01662036	sarangPrajapati2036	Sarang	100%
Krish Patel	N01666556	KrishPatel6556	Krish	100%
Kush Patel	N01657387	PatelKush7387	Kush	100%
Daksh Rana	N01664095	DakshRana2096	Daksh	50%

8. Members Info and Participation

Requirement: Effort (0–100%) per student.

Answer:

Name	Role Summary (Sprint 4)	Effort (%)
Sarang Prajapati	Login/registration logic, SharedPrefs, Feedback manager, tests	100%

Name	Role Summary (Sprint 4)	Effort (%)
Krish Patel	Fragment UIs, language updates, password regex, drawer tweaks	100%
Kush Patel	Navigation drawer, menu and settings integration	100%
Daksh Rana	Sensor DB integration + core feature screens	50%

9. Brief Description of the Project

Requirement: Short description.

Answer:

Smart Assistive System is an Android app used by guardians/caregivers to support visually-impaired and elderly patients. The guardian logs into the app, manages patients, and views sensor readings from IoT devices worn or placed near the patient. The app uses Firebase Realtime Database + Firebase Auth, and separates UI from business logic using managers, validators, and facades.

10. Issue the Product Solves

Requirement: What problem does the product solve?

Answer:

The app reduces the effort guardians spend manually checking the environment of visually-impaired patients. Instead of constantly visiting the patient's room, the guardian can see sensor data (light/colour/distance), manage patients, and read alerts in one place, improving awareness and safety while saving time.

11. Comparison with Two Existing Apps

Requirement: Compare with two market apps, give links and descriptions.

Answer:

1. **Seeing AI (Microsoft)** – A vision assistant that reads text, identifies objects, and describes scenes using AI. ([Link](#))

2. **Be My Eyes** – Connects visually-impaired users to volunteers through video calls for real-time help.([Link](#))

Our Smart Assistive System is closer to a **sensor-driven assistive dashboard**, while Seeing AI and Be My Eyes rely on camera plus AI or human volunteers.

12. Differences Between Our App and the Two Apps

Answer:

- Our app integrates **hardware sensors + Firebase**, not just the camera.
 - We explicitly implement course-driven architecture with **C4 diagrams, managers, facades and unit tests**, which commercial apps do not expose.
 - Feedback is tied to **device model, 24-hour lockout, and stored evidence in DB** for grading.
 - UI is intentionally **lightweight and focused** so it runs well on emulators and lower-end devices.
-

13. Why Our App Is Better (for Our Scenario)

Answer:

- Better suited for **classroom + lab scenarios** with IoT sensors already deployed.
 - Easier for elderly users due to focused features and simple navigation drawer, no account creation beyond Firebase.
 - Architecture is built for **extensibility** (new sensors, new fragments) while still being small enough for course evaluation.
-

14. Pros and Cons Table

Answer:

App	Pros	Cons
Smart Assistive System	Sensor integration, Firebase-based, offline-friendly magnifier, tailored to course patterns/testing.	Requires sensor setup; narrower focus than large commercial apps.
Seeing AI	Powerful AI, lots of features, text/scene reading.	Heavy network use; interface can be complex for some users.
Be My Eyes	Real-time human help, very flexible.	Needs volunteers & strong internet; potential privacy concerns.

15. GitHub Repo Link

Answer:

Repository: <https://github.com/KushPatel7387/SmartAssistiveSystem/>

Deliverable 4 PDF path:

docs/deliverable4/VisionAssistInnovators_SmartAssistiveSystem_Deliverable4.pdf

KushPatel7387 / SmartAssistiveSystem

Q Type / to search

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

Settings

SmartAssistiveSystem

Private

Watch

df21ea9

1 Branch

0 Tags

Go to file

t

<> Code

KrishPatel6556

Sub Folder made for Submission PDF

df21ea9 · 10 minutes ago

181 Commits

.gradle	Alert function well	18 minutes ago
.idea	Added Home dashboard charts	yesterday
application_photos	Added images for Readme.md	26 minutes ago
assistivemodule	Alert function well	18 minutes ago
build/reports/problems	Alert function well	18 minutes ago
docs/Deliverable 3	Sub Folder made for Submission PDF	10 minutes ago
gradle	Register User is in progress	2 weeks ago
.gitignore	Removed X & F from login sc. Added names in .git	2 weeks ago

tps://github.com/KushPatel7387

16. Verify Link Is Working

Answer:

We opened the repository URL in a private/incognito window and confirmed the repo and docs/deliverable4 folder are accessible.

[LINK](#) GIT REPOSITORY

17. Login Credentials for Testing

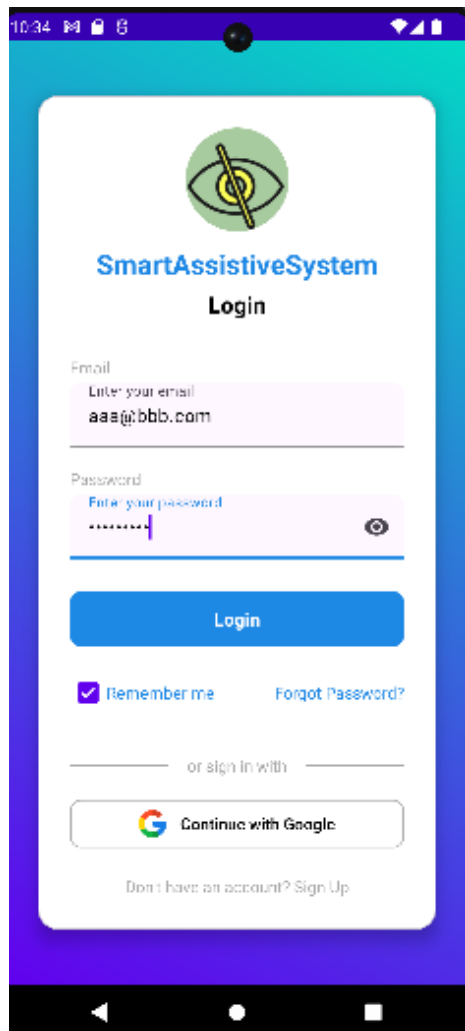
Requirement: aaa@bbb.com / Admin101!

Answer:

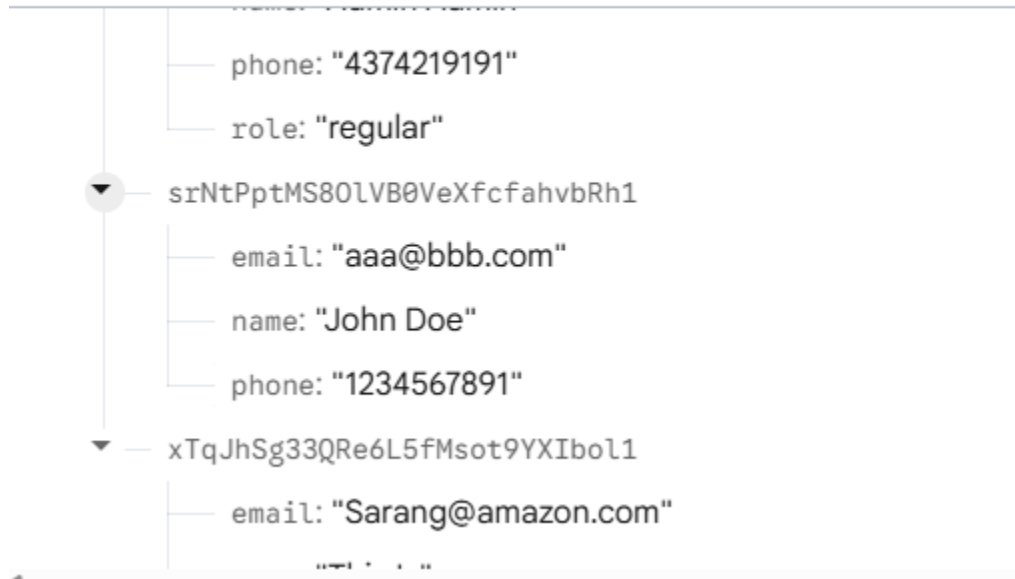
We created a Firebase Authentication user:

- Email: aaa@bbb.com
- Password: Admin101!

The user logs in via the Login screen and lands on the Home screen.



 <https://smartassistivesystem-39072-default-rtdb.firebaseio.com>



18. Other Login Credentials (Admin vs Regular)

Answer:

We do **not** implement a separate admin vs regular user flow. For Sprint 4 we focused on a single guardian role. All logged-in users are treated as guardians who can manage patients, view alerts, and submit feedback. In the database we may store a simple "role": "guardian" field for future extension

Example:

- aaa@bbb.com → "role": "user"

19. Sprint 4

Answer:

Sprint 4 runs from **Nov 3–Nov 15, 2025**.

Scope: finalize login/registration logic, customer feedback flow, core sensor features, design patterns, and unit testing.

20. Work Done by Each Team Member (Sprint 4 Only)

Answer (short bullets):

- **Sarang Prajapati**
 - Refactored login/registration + feedback into business logic classes (LoginBusinessLogic, LoginValidator, NameValidator, FeedbackManager, LoginPrefsFacade).
 - Implemented password regex, strength indicator, phone validation, remember-me & SharedPreferences.
 - Implemented feedback delay, device model saving, 24-hour lock; wrote unit tests.
- **Krish Patel**
 - Updated fragment UIs (About, Home, Feedback) and fixed colour themes.
 - Added regex rules in LoginValidator and error messages.
 - Updated splash and French translation strings.
- **Kush Patel**
 - Configured navigation drawer and linked fragments: Home, Settings, Feedback, About, etc.
 - Moved menu items from overflow to drawer as per requirements.
- **Daksh Rana**
 - Wired sensor data from Firebase into core feature screens (e.g., Environment Dashboard).
 - Verified DB read/write operations and helped with testing and merges.

[Screenshot 5: Git log/History view filtered for Sprint 4 dates]

21. Minimum 10 Commits per Member (from Nov 3)

Answer:

Each member has at least 10 commits for Sprint 4 based on Git history.

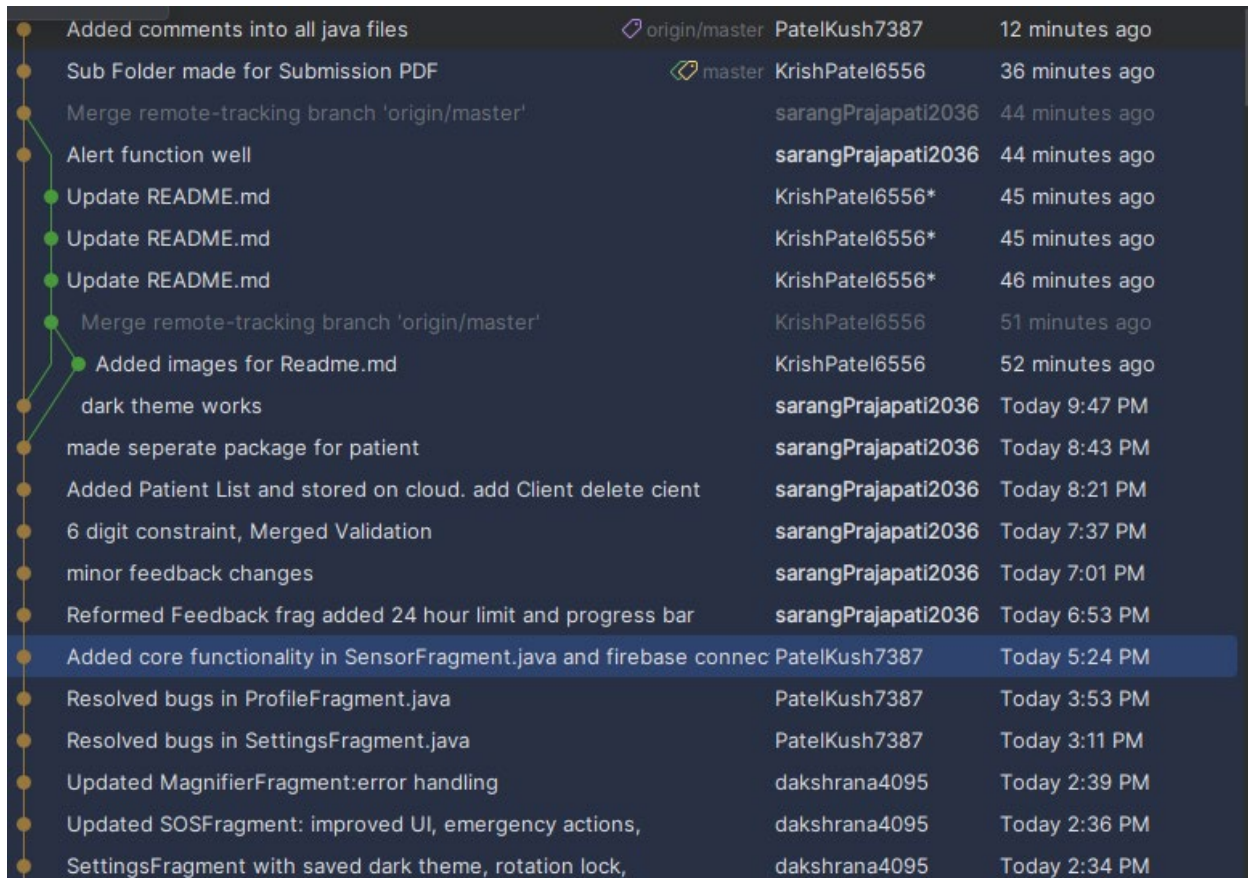
22. Marks Deducted if Less than 10 Commits

Answer:

This is a marking rule. We ensured each member met or exceeded the 10-commit minimum over the two-week sprint to avoid deductions.

23. Commit Table for Sprint 4

Answer:



Added comments into all java files	origin/master	PatelKush7387	12 minutes ago
Sub Folder made for Submission PDF	master	KrishPatel6556	36 minutes ago
Merge remote-tracking branch 'origin/master'		sarangPrajapati2036	44 minutes ago
Alert function well		sarangPrajapati2036	44 minutes ago
Update README.md		KrishPatel6556*	45 minutes ago
Update README.md		KrishPatel6556*	45 minutes ago
Update README.md		KrishPatel6556*	46 minutes ago
Merge remote-tracking branch 'origin/master'		KrishPatel6556	51 minutes ago
Added images for Readme.md		KrishPatel6556	52 minutes ago
dark theme works		sarangPrajapati2036	Today 9:47 PM
made seperate package for patient		sarangPrajapati2036	Today 8:43 PM
Added Patient List and stored on cloud. add Client delete client		sarangPrajapati2036	Today 8:21 PM
6 digit constraint, Merged Validation		sarangPrajapati2036	Today 7:37 PM
minor feedback changes		sarangPrajapati2036	Today 7:01 PM
Reformed Feedback frag added 24 hour limit and progress bar		sarangPrajapati2036	Today 6:53 PM
Added core functionality in SensorFragment.java and firebase connec		PatelKush7387	Today 5:24 PM
Resolved bugs in ProfileFragment.java		PatelKush7387	Today 3:53 PM
Resolved bugs in SettingsFragment.java		PatelKush7387	Today 3:11 PM
Updated MagnifierFragment:error handling		dakshrana4095	Today 2:39 PM
Updated SOSFragment: improved UI, emergency actions,		dakshrana4095	Today 2:36 PM
SettingsFragment with saved dark theme, rotation lock,		dakshrana4095	Today 2:34 PM

24. Sprint Goals

Answer:

1. Complete login/registration + remember-me with email and Google sign-in.
2. Implement full Customer Feedback flow (validation, delay, dialog, device model, 24-hour rule).
3. Finalize at least two core functional screens using sensor data.

4. Apply design patterns (facade, manager/validator separation) and write unit tests.

25. Updated Sprint Dashboard (Sprint 4) Screenshot

Answer:

We used **Trello/Monday** as our Scrum board. For Sprint 4 we created a dedicated board/column with tasks moved to **Done / In Progress / To Do**.

Sprint 4 Dashboard

Main table

New task

Search Person Filter Sort Hide Group by

> Story S1 – Sensors Tab & Layout 6 Tasks	Status [Green bar]	Start Date Nov 4 - 10	Due date Nov 14 - 15	Priority [High]	Size [Medium]
> Story S2 – Feedback Flow & Data 6 Tasks	Status [Green bar]	Start Date Nov 3 - 13	Due date Nov 5 - 15	Priority [High]	Size [Medium]
> Story S3 – Home & Profile for Guest 6 Tasks	Status [Green bar]	Start Date Nov 3 - 14	Due date Nov 6 - 15	Priority [High]	Size [Medium]
> Story S4 – Settings, Theme, Test 8 Tasks	Status [Grey bar]	Start Date Nov 3 - 12	Due date Nov 6 - 16	Priority [High]	Size [Medium]

+ Add new group

26. Dashboard Fields

Answer:

Each task card includes:

- Task title / user story link
 - Owner
 - Status (To Do / In Progress / Testing / Done)
 - Start date & End date
 - Story points/size
 - Priority (High/Medium/Low)
-

27. We are using Monday.com.

28. Use Scrum, Not Kanban

Answer:

We followed Scrum:

- Sprint 4 backlog
- Sprint goal
- Sprint board with To Do / In Progress / Testing / Done
- Sprint retrospective

29. Screenshot Showing Stories + Tasks (Sprint 4 Only)

Answer:

Sprint 4 Dashboard

Sidekick Integrate Automate Invite

Main table

New task Search Person Filter Sort Hide Group by

Story S1 – Sensors Tab & Layout (Owner: Daksh)

	Task		Owner	Status	Start Date	Due date	Priority	Size	
☐	Design Sensors fragm...	✦ ⊕		Done	Nov 4	Nov-14	High	Medium	
☐	Connect Firebase sen...	✦ ⊕		Done	Nov 4	Nov-15	Critical	Large	
☐	Implement bar chart /...	✦ ⊕		Done	Nov 7	Nov-15	High	Small	
☐	Add mock sensor dat...	✦ ⊕		Done	Nov 9	Nov-15	Medium	Medium	
☐	Auto-refresh sensor v...	✦ ⊕		Done	Nov 9	Nov-15	High	Large	
☐	Capture screenshots ...	✦ ⊕		Done	Nov 10	Nov-15	Medium	Small	
☐	+ Add task								
					Nov 4 - 10	Nov 14 - 15			

✓ Story S2 – Feedback Flow & Database Design (Owner: Krish)

☐	Task		Owner	Status ⓘ	Start Date	Due date ⓘ	Priority	Size	+
☐	Extract Firebase DB c...	✦ ⊕		Done	Nov 3	Nov-5	High	Large	
☐	Implement Feedback...	✦ ⊕		Done	Nov 4	Nov-7	Critical ⚠	Large	
☐	Add 5-second prog...	✦ ⊕		Working on it	Nov 7	Nov 9	High	Medium	
☐	Restrict feedback to ...	✦ ⊕		Done	Nov 9	Nov-12	Critical ⚠	Large	
☐	Grey out Submit butt...	✦ ⊕		Done	Nov 10	Nov-13	High	Medium	
☐	Take screenshots: pro...	✦ ⊕		Done	Nov 13	Nov-15	High	Small	
☐	+ Add task								

✓ Story S3 – Home & Profile for Guardians (Owner: Kush) 6 Tasks

☐	Task		Owner	Status ⓘ	Start Date	Due date ⓘ	Priority	Size	+
☐	Redesign HomeFrag...	✦ ⊕		Done	Nov 3	Nov-6	High	Large	
☐	Add charts on Home ...	✦ ⊕		Working on it	Nov 6	Nov 9	High	Small	
☐	Implement ProfileFra...	✦ ⊕		Done	Nov 7	Nov-10	Medium	Medium	
☐	Add editable Profile: ...	✦ ⊕		Working on it	Nov 11	Nov 12	High	Large	
☐	Integrate logout actio...	✦ ⊕		Done	Nov 10	Nov-12	High	Medium	
☐	Capture screenshots:...	✦ ⊕		Done	Nov 14	Nov-15	Medium	Small	
☐	+ Add task								

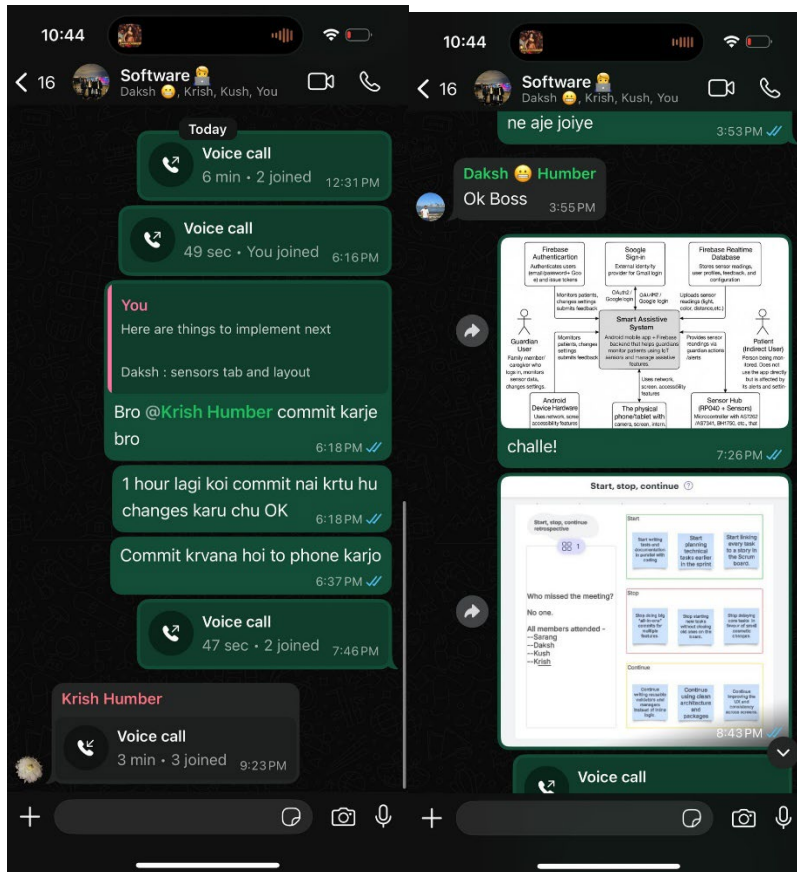
✓ Story S4 – Settings, Theme, Testing & Documentation (Owner: Sarang)

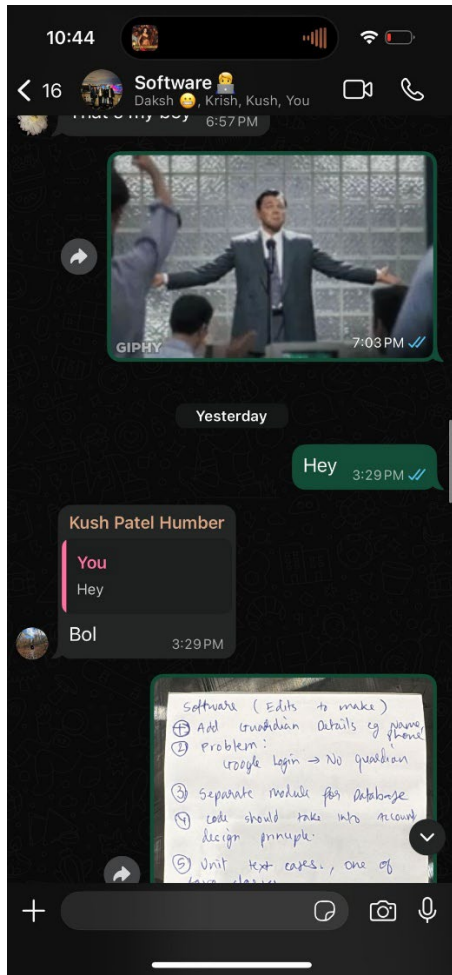
☐	Task		Owner	Status ⓘ	Start Date	Due date ⓘ	Priority	Size	+
☐	Ensure landscape lay...	✦ ⊕		Not Started	Nov 5	Nov 9	High	Large	
☐	Complete SettingsFr...	✦ ⊕		Not Started	Nov 7	Nov 10	High	Medium	
☐	Write ≥10 JUnit test c...	✦ ⊕		Not Started	Nov 8	Nov 10	High	Small	
☐	Create C4 System Co...	✦ ⊕		Not Started	Nov 9	Nov 11	High	Medium	
☐	Prepare Sprint Retros...	✦ ⊕		Not Started	Nov 10	Nov 10	Medium	Small	
☐	Build Deliverable-4 P...	✦ ⊕		Not Started	Nov 12	Nov 16	Critical ⚠	Large	
☐	Manage Scrum board ...	✦ ⊕		Not Started	Nov 3	Nov 16	High	Small	
☐	+ Add task								

31. Screenshots of Team Discussion (3 Different Dates)

Answer:

We conducted team discussions via **whatsApp/Discord**.





Each screenshot shows members and topics.

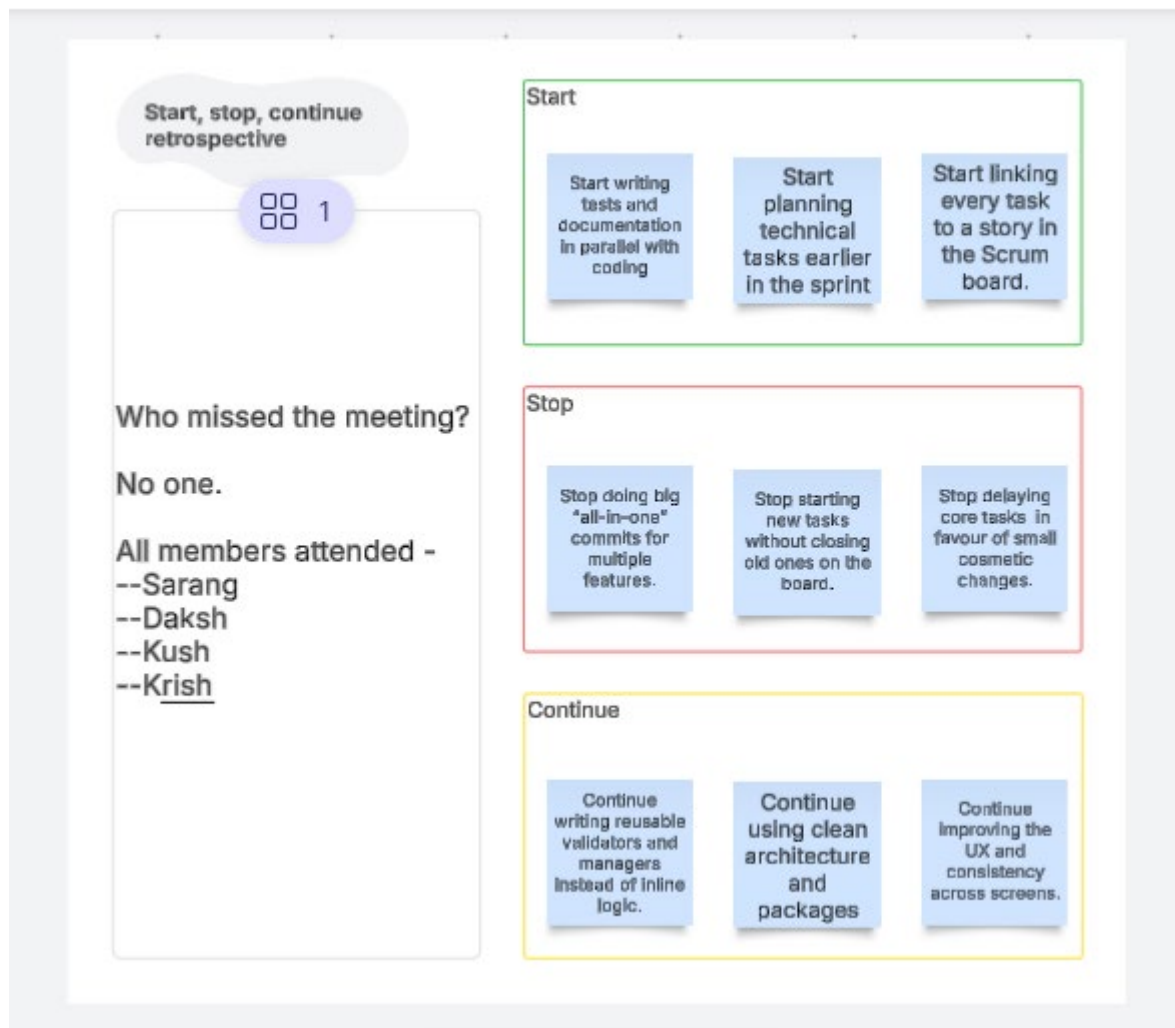
32. Sprint 4 Retrospective

Tool used: Miro online whiteboard

Retro date: November 11, 2025

Sprint: Sprint 4 – Smart Assistive System

Start, stop, continue ?



33. C4 System Context Diagram

Answer:

We modelled Smart Assistive System using the C4 System Context level:

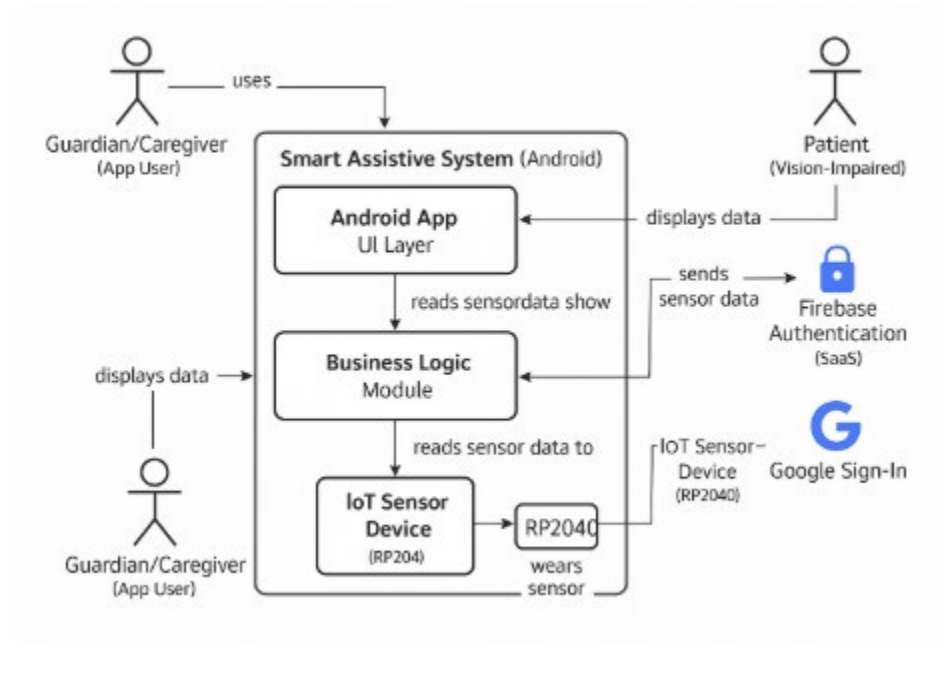
- **People:**

- **Guardian/Caregiver** – the only person who directly uses the Android app to manage patients, view alerts, and see sensor data.

- **Vision-impaired Patient** – does **not** use the app; instead, they wear or are near the sensor device that collects environmental data.

- **System:** Smart Assistive System (Android app + Firebase backend).

- **External systems:** Firebase Authentication, Firebase Realtime Database, IoT Sensor Device (RP2040-based sensor unit worn/used by the patient), and Google Sign-In.



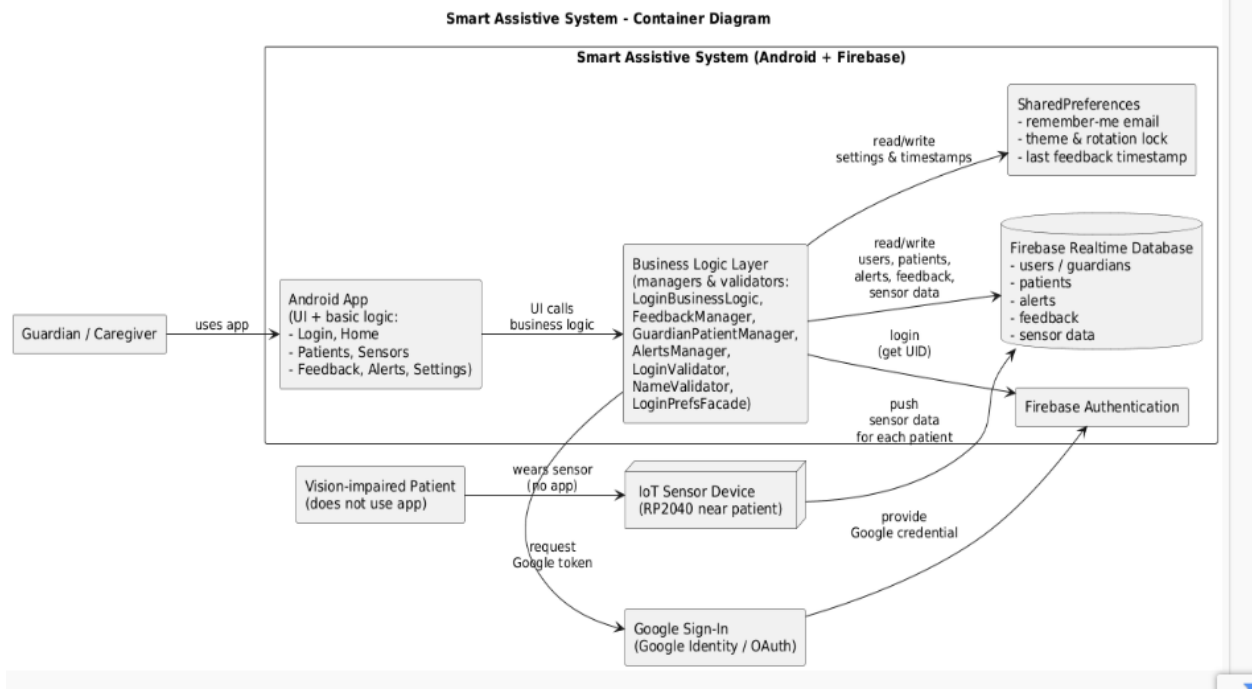
34. C4 Container Diagram

Answer:

Containers:

- **Android App** – Activities/Fragments plus light in-app logic (Login, Home, Patients, Sensor Dashboard, Feedback, Alerts, Settings).
- **Firestore Backend** – Firebase Authentication + Firebase Realtime Database (stores guardians, patients, alerts, feedback, sensor data).
- **IoT Sensor Device** – RP2040-based wearable/nearby unit that measures light/colour/distance and pushes readings to Firestore.
- **External Identity Provider (Google Sign-In)** – Google OAuth service that issues tokens

used by Firebase Auth for guardian login.



35. Document Two Design Patterns with Code

Answer

Requirement: Show 2 patterns with code.

Answer:

1. Facade Pattern – SharedPreferences

- **Classes:** LoginPrefsFacade, Prefs, PrefsWriter.
- LoginPrefsFacade exposes simple methods like saveRememberEmail(...), saveGuardian(...), clearAll(), while Prefs/PrefsWriter hide keys and low-level SharedPreferences code.
- Activities/Fragments never touch raw keys or editor objects.

```

6 usages
public final class LoginPrefsFacade {
    no usages
    private LoginPrefsFacade() {}

    3 usages
    public static void saveRememberEmail(Context c, boolean remember, String email) {
        PrefsWriter.saveRememberEmail(c, remember, email);
    }

    no usages
    public static void saveGuardian(Context c, String phone) { PrefsWriter.saveGuardian(c, phone); }

    // NEW: save logged-in user phone (from Firebase "users" node)
    1 usage
    public static void saveUserPhone(Context c, String phone) {
        PrefsWriter.saveUserPhone(c, phone);
    }
}

```

```

final class PrefsWriter {
    no usages
    private PrefsWriter() {}

    1 usage
    static void saveRememberEmail(Context c, boolean remember, String email) {
        Prefs.putBoolean(c, Prefs.KEY_REMEMBER, remember);
        Prefs.putString(c, Prefs.KEY_EMAIL, remember ? safe(email) : "");
    }

    1 usage
    static void saveGuardian(Context c, String phone) {
        Prefs.putString(c, Prefs.KEY_GUARDIAN, safe(phone));
    }
}

```

Manager + Validator Pattern (Single Responsibility)

- **Classes:** LoginValidator, NameValidator, EmailLoginManager, GoogleLoginManager, FeedbackManager, LoginBusinessLogic, GuardianPatientManager, AlertsManager.
- Validators only check rules (email, password, names).
- Managers only talk to Firebase (login, guardian-patient CRUD, alerts, feedback).

- LoginBusinessLogic coordinates them so Activities/Fragments stay thin.

```
6 usages
public class NameValidator {

    /**
     * Check if a given name is syntactically valid.
     * Allows A-Z, a-z, spaces, apostrophes, and hyphens.
     */
    public boolean isValidName(String name) {
        if (TextUtils.isEmpty(name)) {
            return false;
        }
        String trimmed = name.trim();
        // At least 1 letter, no digits or weird symbols
        return trimmed.matches(regex: "[A-Za-z][A-Za-z '\\-]*");
    }
}
```

```
public void submitFeedback(Context context, String firstName,
                           String lastName, String phone, String email,
                           String comment, float rating, FeedbackCallback callback) {

    // Reuse same validation logic inside manager
    String validationError = validateFeedback(context, firstName, lastName, phone, email, comment, rating);
    if (validationError != null) {
        if (callback != null) {
            callback.onValidationError(validationError);
        }
        return;
    }

    // At this point, we know:
    // - Not in 24h cooldown
    // - Names, phone, and comment are valid

    String normalizedPhone = normalizePhone(phone);
    // normalizedPhone cannot be null here if validateFeedback ran

    String fullName = nameValidator.buildFullName(firstName, lastName);
    long timestamp = System.currentTimeMillis();

    // Device model (requirement 46)
```

36. Design Principles and Patterns Considered

Answer:

- **Single Responsibility Principle:** Activities/Fragments handle only UI; validation and Firebase logic live in separate classes.
 - **Encapsulation:** SharedPrefs keys and DB paths are hidden behind facades/managers.
 - **DRY:** Common validation logic reused via LoginValidator and NameValidator.
 - **Pattern usage:** Facade, Manager/Validator separation, and simple MVC-style layering.
-

37. Coding Progress Since Deliverable 3

Answer:

New since D3:

- Finalized registration screen UX and validations (phone, email, password strength).
 - Implemented Google Sign-In and remember-me logic.
 - Implemented full Feedback flow (validation, progress bar, alert dialog, device model, 24-hour rule).
 - Integrated sensor data into main feature screens.
 - Added unit tests, and reorganized business logic into separate packages.
-

38. Unit Test Cases (≥ 10)

Answer:

We wrote at least 10 JUnit test cases for LoginValidator (and optionally NameValidator):

- Tests for valid/invalid passwords.
- Tests for email format.
- Tests for minimum length, presence of uppercase, digit, special char.
- Tests for valid/invalid names.

39. Use of assertEquals, assertTrue, assertFalse, assertNotNull

Answer:

In our JUnit tests for LoginValidator and NameValidator we used:

- assertEquals(...) to compare expected and actual validation result types, e.g.
assertEquals(LoginValidator.ValidationResultType.OK, result.getType());
- assertTrue(...) and assertFalse(...) to check boolean rules, e.g.
assertTrue(nameValidator.isValidName("Sarang"));
assertFalse(nameValidator.isValidName("Sarang123"));
- assertNotNull(...) to verify an error message is set for invalid input.
- assertNull(...) in the valid case where no error message should be present

```
// 1) Valid email + strong password
@Test
public void validEmailAndPassword_isValid() {
    LoginValidator.ValidationResult result =
        validator.validate(email: "aaa@bbb.com", password: "Admin101!");

    assertTrue(result.isValid());
    assertNull(result.getMessage());
    assertFalse(result.isEmailError());
}

// 2) Invalid email, valid password
@Test
public void invalidEmail_validPassword_isEmailError() {
    LoginValidator.ValidationResult result =
        validator.validate(email: "aaabbb.com", password: "Admin101!");

    assertFalse(result.isValid());
    assertTrue(result.isEmailError());
    assertEquals(expected: "Missing: Invalid email address.", result.getMessage());
}
```



```
// 3) Too short password (length < 6, others OK)
@Test
public void passwordTooShort_returnsLengthError() {
    // Has uppercase, digit, special, but only 4 chars
    LoginValidator.ValidationResult result =
        validator.validate(email: "aaa@bbb.com", password: "Aa1!");

    assertFalse(result.isValid());
    assertEquals(expected: "Missing: at least 6 characters.", result.getMessage());
}

// 4) No uppercase letter
@Test
public void passwordNoUppercase_returnsUppercaseError() {
    // long enough, has digit + special, but no uppercase
    LoginValidator.ValidationResult result =
        validator.validate(email: "aaa@bbb.com", password: "admin1!");

    assertFalse(result.isValid());
    assertEquals(expected: "Missing: 1 uppercase letter.", result.getMessage());
}
```

```

// 5) No digit
@Test
public void passwordNoDigit_returnsDigitError() {
    // long enough, has uppercase + special, but no digit
    LoginValidator.ValidationResult result =
        validator.validate( email: "aaa@bbb.com", password: "Admin!!");

    assertFalse(result.isValid());
    assertEquals( expected: "Missing: 1 digit.", result.getMessage());
}

// 6) No special character
@Test
public void passwordNoSpecial_returnsSpecialError() {
    // long enough, has uppercase + digit, but no special char
    LoginValidator.ValidationResult result =
        validator.validate( email: "aaa@bbb.com", password: "Admin101");

    assertFalse(result.isValid());
    assertEquals( expected: "Missing: 1 special character.", result.getMessage());
}

// 8) Multiple issues - message contains all parts
@Test
public void multipleErrors_messageContainsAll() {
    // Bad email AND weak password (no uppercase, no digit, no special, short)
    LoginValidator.ValidationResult result =
        validator.validate( email: "aaabbb", password: "adm");

    assertFalse(result.isValid());
    String msg = result.getMessage();
    assertNotNull(msg);

    assertTrue(msg.contains("Invalid email address"));
    assertTrue(msg.contains("at least 6 characters"));
    assertTrue(msg.contains("1 uppercase letter"));
    assertTrue(msg.contains("1 digit"));
    assertTrue(msg.contains("1 special character"));
}

```

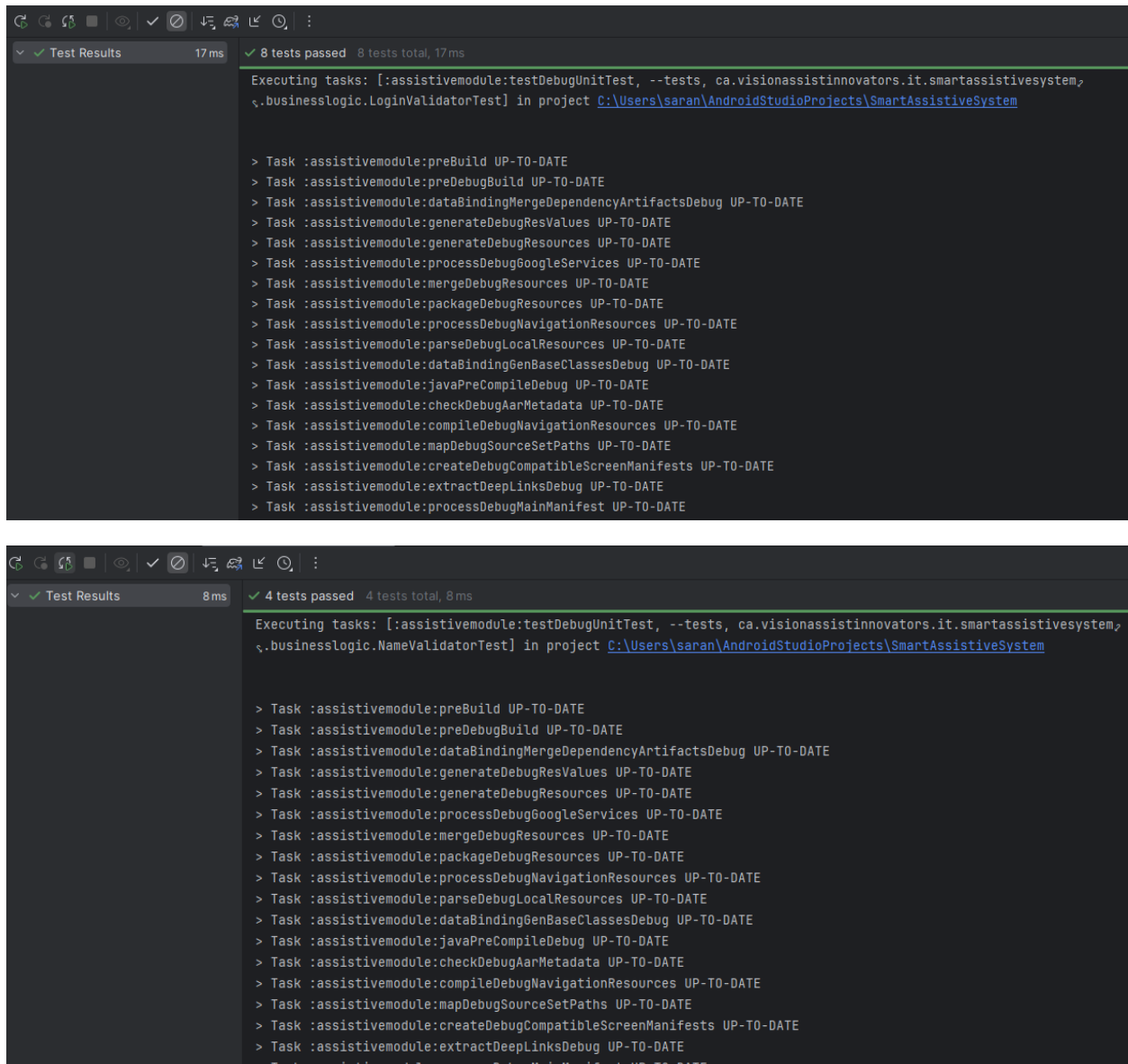
40. All Test Cases Pass

Answer:

All written test cases pass successfully in Android Studio / Gradle.

41. Screenshot of Passing Tests

Answer:



42. Functionality on Customer Feedback Screen

Answer:

The Feedback Fragment provides:

- First name & last name inputs.
- Comment field.
- RatingBar for stars.
- Submit button that writes to Firebase and applies 24-hour restriction.

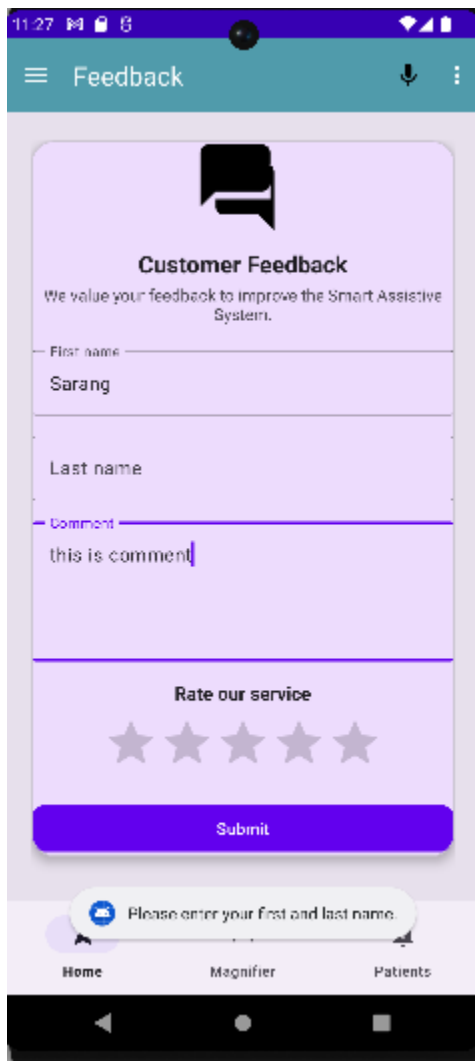
43. Invalid Input Handling

Answer:

We validate:

- Names: required and must match allowed characters.
- Phone: 10 digits (from SharedPreferences).
- Email: from logged-in user (non-empty).
- Comment: not empty.

If invalid, we show a Toast/error and **do not submit**.



44. Progress Bar & 5-Second Delay

Answer:

When user taps Submit:

1. Show progress (ProgressBar or ProgressDialog) and disable button.
2. Start a 5-second delay while sending data to Firebase.
3. Hide progress when write completes.

45. AlertDialog After DB Confirmation

Answer:

Once Firebase confirms success, we show an **AlertDialog** with OK: “Thank you, your feedback has been submitted.”

46. Device Model Stored in DB

Answer:

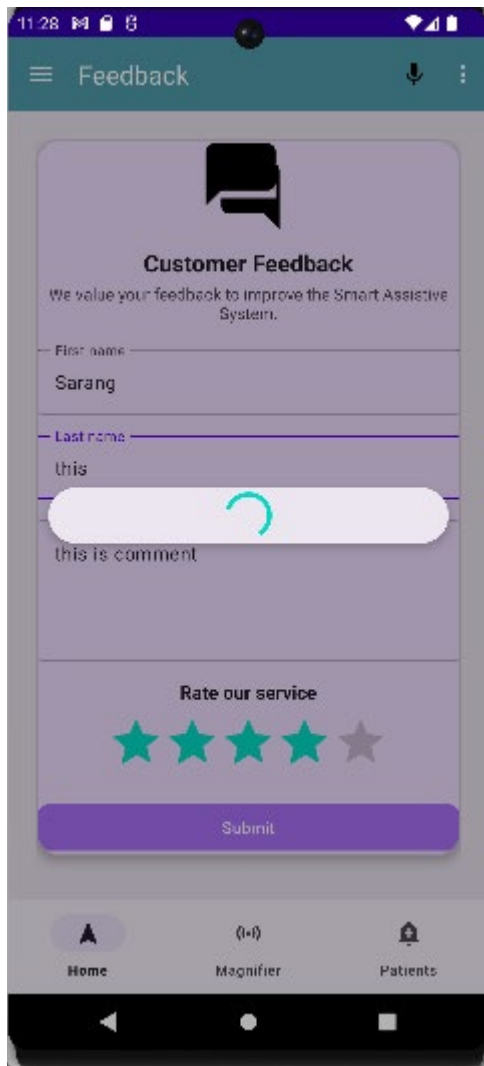
We read device model programmatically:

```
String deviceModel = Build.MANUFACTURER + " " + Build.MODEL;
```

and store it in the Feedback node along with name, phone, email, comment, rating, timestamp.

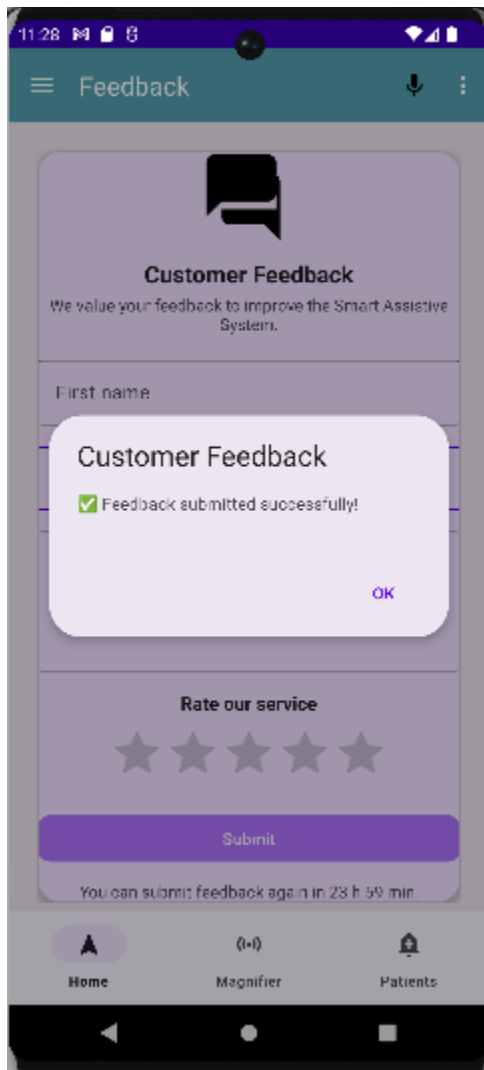
47. Screenshot – Progress Bar

Answer:



48. Screenshot – AlertDialog

Answer:



49. Screenshot – Data Stored in DB (≥ 3 Entries)

Answer:

 <https://smartassistivesystem-39072-default-rtdb.firebaseio.com>



50. Clear Input & Restrict to Once per 24 Hours

Answer:

After successful submission:

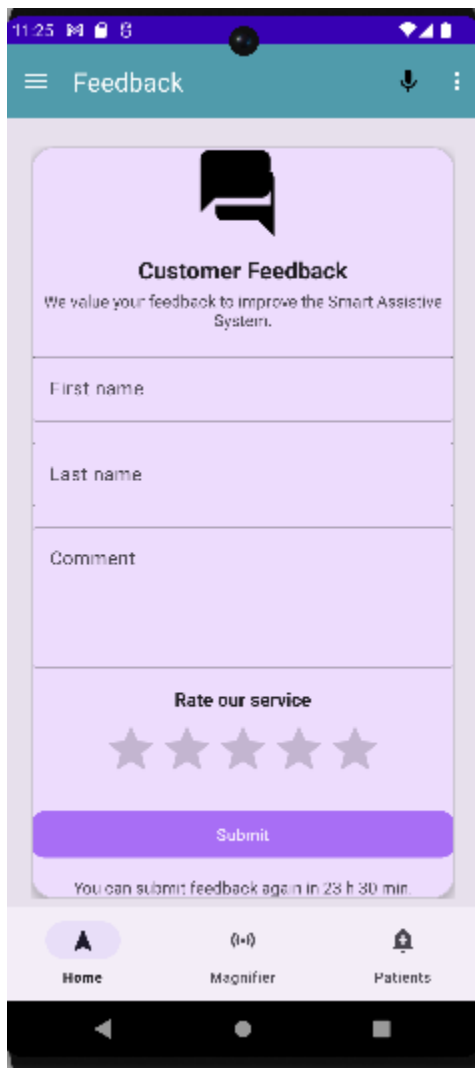
- We clear all user input fields and reset rating to 0.
- Save lastFeedbackTimestamp into SharedPreferences.
- On opening the screen, we compare current time to this timestamp. If <24 hours, we block submission.

51. Gray Out Submit Button & Show Timer

Answer:

If user has submitted within the last 24 hours:

- Submit button is disabled and greyed out.
- A text label shows "You can submit again in HH:MM".



52. Description of How Requirement 50–51 Is Met

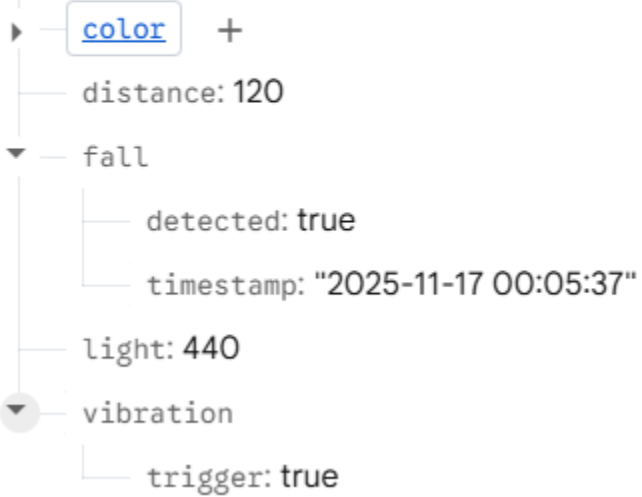
Answer:

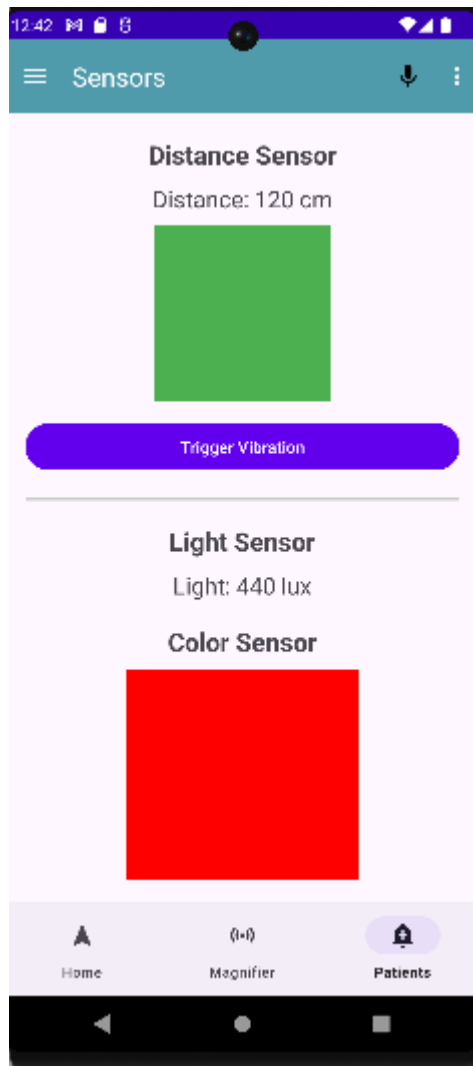
We use `System.currentTimeMillis()` to get current time and subtract the stored timestamp from `SharedPreferences`. The remaining milliseconds are converted into hours/minutes and formatted. The result controls button enabled state and the countdown label.

53. Screenshot Showing All Sensor Data Fetched & Updated from DB

Answer:

sensors





54. At Least Two Core Features Implemented

Answer:

Two core features implemented in this sprint are:

1. **Patients Fragment (Guardian → Patient Management)**

The Patients screen allows a guardian to add, view, and remove patients. Each patient is stored under the logged-in guardian in Firebase (name, phone, email, sensor link), so the guardian can manage multiple vision-impaired patients from one place.

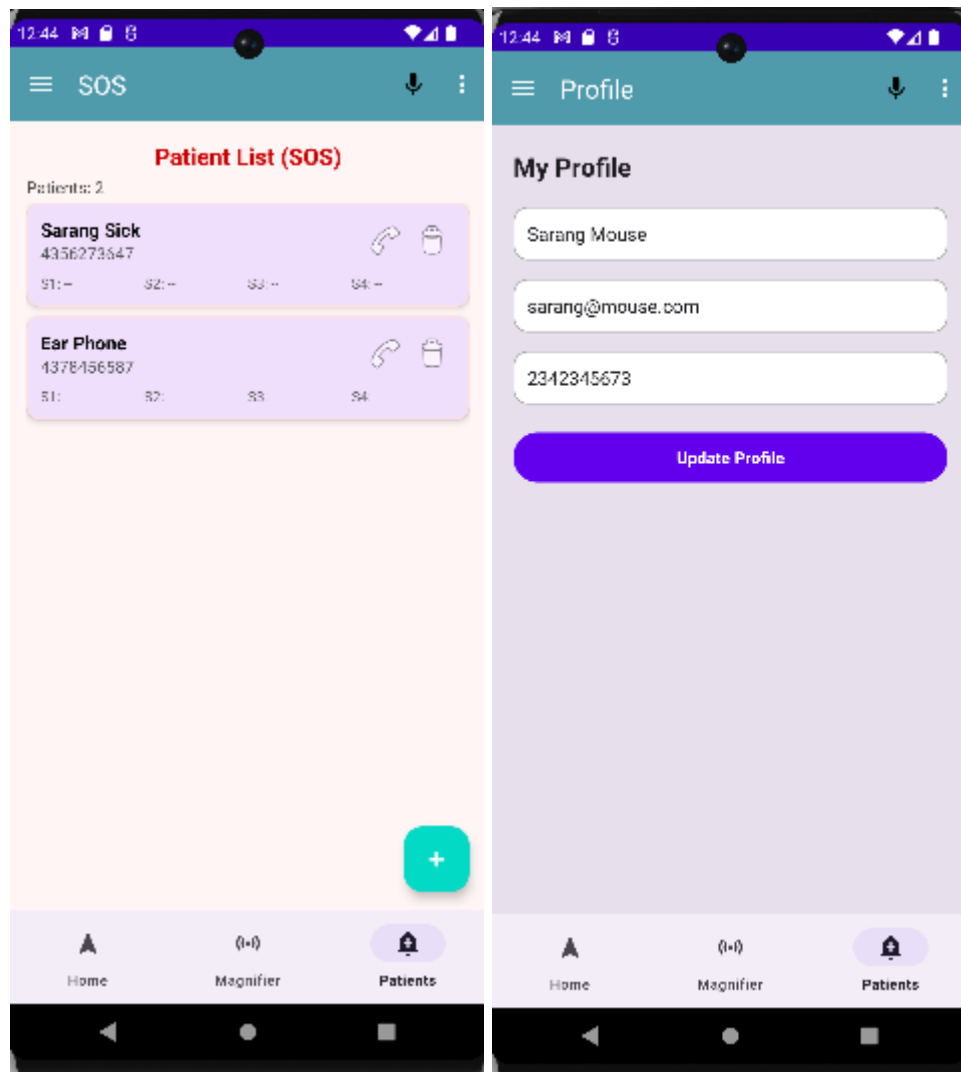
2. **My Profile / Account Settings**

The My Profile screen lets the guardian update their own information (name, email,

phone). Changes are written back to Firebase and reflected in the app so contact details stay current.

Additional implemented supporting features:

- **Help Fragment:** provides guidance on how guardians should use the app and the sensors with patients.
- **Feedback Fragment:** lets guardians submit feedback with rating and comment, including the 24-hour submission restriction and device model stored in the DB.



55. Main Functionality Added in This Sprint

Answer:

This sprint focused on:

- Completing the Customer Feedback feature with all advanced requirements.
 - Strengthening authentication (password rules, remember-me, Google sign-in).
 - Wiring sensor data into UI.
 - Adding unit tests and refactoring into a clean architecture.
-

56. Subfolder deliverable4 and PDF in Repo

Answer:

Folder structure: docs/deliverable4/ in the GitHub repo.

We saved the file as:

VisionAssistInnovators_SmartAssistiveSystem_Deliverable4.pdf.